Projet transversal

"Imaginer le Numérique Malagasy de 2035"

Cahier des charges & exigences algorithmiques & Réalisation & Présentation

ESMIA INNOVATION

Objectif général (thème global)

Thème imposé

Imaginer le Numérique Malagasy de 2035.

Finalité

Produire un **système numérique cohérent** (vision + architecture + prototype), capable de fonctionner dans des contraintes réalistes (réseau, énergie, coûts, accessibilité), tout en démontrant une **maîtrise avancée** des algorithmes et structures de données.

Important

Aucune orientation sectorielle n'est fournie.
Chaque étudiant définit **son propre problème** et **son propre périmètre** dans le cadre du thème.

Règles du jeu (sans orientation)

- **Exigence** Chaque étudiant choisit librement : le domaine, les utilisateurs, les données, l'interface, les contraintes.
- **Exigence** Le projet doit inclure : **(i)** une architecture, **(ii)** un modèle de données, **(iii)** un prototype exécutable, **(iv)** des algorithmes avancés justifiés.
- **Exigence** Interdiction de livrer une simple application CRUD sans composante algorithmique/optimisation.
- **Exigence** Toute fonctionnalité majeure doit être justifiée par au moins **une structure de données** et **un algorithme** adaptés.

Livrables attendus

Livrables techniques

- **Livrable** Prototype (web / mobile / desktop / API) + instructions d'exécution.
- **Livrable** Code source versionné (Git) + historique de commits.
- **Livrable** Jeu de données (réel, synthétique ou collecté) + description.

Livrables d'ingénierie

- **Livrable** Cahier des charges (problème, périmètre, utilisateurs, contraintes, cas d'usage).
- **Livrable** Dossier d'architecture (UML : use-case + classes + séquence ; schéma BD ; diagramme de déploiement).
- **Livrable** Dossier algorithmique (choix, complexités, preuves/arguments, tests).
- **Livrable** Pitch (5 min) + démonstration (5 min).

Contraintes minimales (réalisme 2035)

Chaque étudiant **déclare et justifie** au moins 3 contraintes parmi :

- Connectivité intermittente
- Faible débit
- Coupures d'énergie
- Coût d'infrastructure
- Accessibilité (handicap)
- Multilingue (FR/MG)
- Sécurité/Confidentialité
- Montée en charge (scalabilité)
- Temps réel / latence
- Données bruitées/incomplètes

Exigence

Les contraintes déclarées doivent se traduire par des **choix d'architecture** et des **choix algorithmiques**.

Exigences algorithmiques (obligatoires)

Exigence Algorithmes avancés *(minimum 2 familles)*

Chaque projet doit intégrer **au moins 2** familles parmi :

- **Graphes / Réseaux :** Dijkstra, Bellman–Ford, Floyd–Warshall, Prim, Kruskal, A*.
- **Optimisation :** programmation dynamique, glouton, branch-and-bound (version simplifiée), sac à dos, ordonnancement.
- **Chaînes/Recherche :** KMP, Boyer–Moore, Rabin–Karp.
- **Streaming / Fenêtrage :** top-k, comptage approx., maintien d'agrégats.
- **IA "classique" (optionnel) :** k-NN, arbres de décision (avec justification), sans boîte noire obligatoire.

Interdit

Utiliser une bibliothèque "tout fait" sans implémenter/justifier l'algorithme.

Exigences structures de données (obligatoires)

Exigence Structures avancées *(minimum 3)*

Chaque projet doit utiliser **au moins 3** structures parmi :

- Tas binaire / Tas de Fibonacci (ou équivalent) → priorités.
- Arbres équilibrés (AVL / Red-Black) → indexation.
- Tables de hachage (avec gestion collisions) → accès amorti.
- Union-Find (DSU) → composantes connexes / Kruskal.
- Trie / Radix tree → recherche textuelle.
- Segment tree / Fenwick tree → requêtes intervalle.
- Graphes (listes d'adjacence, matrices) → modélisation.

Exigence

Le dossier doit préciser **pourquoi** la structure est adaptée (complexité, mémoire, contraintes).

Composante optimisation : exigences

Exigence Optimisation mesurable

Le projet doit contenir une optimisation démontrée par :

- un **problème formalisé** (objectif + contraintes),
- une **solution naïve** (baseline),
- une **solution optimisée** (algorithme/structure),
- une **mesure** : temps, mémoire, coût, distance, latence, etc.

Livrable Preuves/arguments attendus

- Analyse de complexité : $O(\cdot)$, cas moyen/pire si pertinent.
- Tests : jeux de données + courbes/mesures + interprétation.

Dossier algorithmique : contenu minimal

Livrable Structure du dossier (plan imposé)

1. Problème(s) algorithmique(s) du projet : définition, entrées/sorties.
2. Modèles : graphes, contraintes, métriques, hypothèses.
3. Algorithmes choisis : description + pseudo-code (synthétique).
4. Structures de données : définition + intégration dans le code.
5. Complexité : temps/mémoire + justification.
6. Validation : tests unitaires + tests de charge (petit protocole).
7. Résultats : tableau comparatif baseline vs optimisé.

Prototype : exigences minimales

Exigence Fonctionnel démontrable

- Au moins **2 parcours utilisateurs** complets en démo.
- Au moins **1 API** ou module principal testable.
- Journalisation (logs) pour expliquer la démo.

Exigence Qualité

- Tests : au moins 5 tests unitaires sur les modules algorithmiques.
- Gestion d'erreurs et cas limites (données manquantes, incohérences).
- Documentation d'exécution : `README` + commande de lancement.

Contraintes d'implémentation (liberté cadrée)

Liberté technologique

Langages, frameworks et SGBD libres, à condition de :

- justifier les choix,
- maîtriser l'implémentation algorithmique (pas de boîte noire).

Recommandation

Préférer une architecture simple : *monolithe modulaire* ou *API + client*.
La complexité attendue est **algorithmique**, pas l'empilement de technologies.

Évaluation (grille)

Évaluation Critères	**Évaluation** Points
Pertinence du problème + cohérence avec “2035” (sans orientation)	15
Architecture + modélisation (UML, BD, déploiement)	20
Algorithmes avancés : pertinence, implémentation, qualité	20
Structures de données avancées : choix, intégration, justification	15
Optimisation mesurée : baseline vs optimisé, protocole de test	15
Qualité du code, tests, robustesse, documentation	10
Pitch + démonstration (clarté, maîtrise, réponses)	5
Total	**100**

Critères de disqualification (à éviter)

- Absence d'algorithmes avancés ou simple utilisation de bibliothèques sans maîtrise.
- Absence de structures de données avancées (ou usage injustifié).
- Prototype non exécutable / non démontrable.
- Dossier algorithmique vide ou non conforme (pas de baseline, pas de mesure).
- Plagiat (code ou documentation).

Organisation & déroulé (modifiable)

Exemple de cadence

- Phase 0 : lancement (brief + règles + formation des étudiants)
- Phase 1 : cadrage (problème, contraintes, périmètre, architecture)
- Phase 2 : implémentation (prototype + modules algorithmiques)
- Phase 3 : optimisation (baseline vs optimisé + mesures)
- Phase 4 : finalisation (tests, dossier, pitch)

Point de contrôle

Un check technique (5 minutes) est prévu : **choix algorithmiques** + **structures** + **baseline**.

Dépôt & nommage (exigences)

Exigence Dépôt Git

- `/docs` : cahier des charges + dossier d'architecture + dossier algorithmique
- `/src` : code source
- `/data` : données (ou script de génération)
- `README.md` : exécution + démo + commandes

Exigence Traçabilité

- Commits réguliers, messages clairs
- Attribution des contributions (membres/parts)

Checklist finale (à cocher avant soumission)

- Problème défini + contraintes déclarées
- UML (use-case, classes, séquence)
- Modèle BD + justification
- 2 familles d'algorithmes avancés
- 3 structures de données avancées
- Baseline implémentée
- Version optimisée implémentée
- Mesures (temps/mémoire/coût) + tableau
- Tests unitaires (min. 5)
- Prototype exécutable + README
- Pitch prêt (5 min) + démo (5 min)

Clôture

Message clé

Le thème est volontairement global.
La réussite dépend de la capacité à : **problématiser**, **modéliser**, **implémenter** et **optimiser**.

Bonne conception, bon code, et surtout : **bonnes justifications** !